Comparing domain specific agent construction strategies

A qualitative comparison of how each approach starts, encodes domain methodology, and accommodates change. Relative speed and flexibility depend on task scope, team experience, and the fit of the chosen framework or base harness.

Dimension	Custom Agentic Application	Harness Specialization	revfactory/harness (harness-rev)	HarnessWeaver
Starting point	Domain problem and expertise; build an application on an agent framework.	An existing general harness, specialized for a domain.	An existing harness repository and its conventions, adapted through project-specific guidance and components.	Domain problem, domain knowledge, and an explicit methodology used to synthesize a harness prototype.
Methodology	Encoded across prompts, workflow code, tools, and application logic.	Translated manually into instructions, skills, tools, and orchestration.	Expressed through harness-specific configuration, instructions, skills, and repository workflows.	Made explicit as an input to generation and iterative refinement.
Domain knowledge	Distributed through prompts, data/RAG, tools, schemas, and code.	Added through domain instructions, skills, resources, and tools.	Added within the conventions and extension points of the chosen harness.	Represented explicitly and translated into harness components.
Harness design	Designed and implemented by the application team.	Designed manually around the base harness.	Constrained by, and adapted to, the selected harness structure.	Generated as an editable, evolvable prototype; design remains open to revision.
Agent architecture	Chosen and implemented directly (e.g., workflow, multi-agent, RAG).	Mostly inherited from the base harness; extensions may add agents or orchestration.	Largely shaped by the selected harness and its supported patterns.	Deferred during prototyping; may later be realized through a custom application or specialized harness.
Runtime/framework	Selected directly (e.g., LangChain/LangGraph or another framework).	The runtime of the chosen base harness.	The runtime/framework of the harness being adapted.	Can be selected or committed to later, subject to available realization paths.
Evaluation	Application-specific tests, task metrics, and human review.	Task evaluation in the base harness, supplemented as needed.	Evaluation supported or added within the repository's workflow.	Prototype evaluation informs iterative changes; no automatic optimality claim.
Evolution	Changes may involve code, prompts, data, and architecture.	Changes are made manually within harness extension points.	Evolution follows the selected harness's conventions and constraints.	Designed to support rapid modification of domain knowledge, methodology, and harness structure.
Portability / reuse	Reusable components are possible; the application may remain bespoke.	Skills or tools may be reusable when compatible with other harnesses.	Reuse is bounded by compatibility with the underlying harness.	Generated components may be reusable where the target realization supports them; portability is not guaranteed.
Architectural commitment	Relatively early: architecture and runtime are built into the application.	Early commitment to a base harness; domain implementation is layered onto it.	Early commitment to the chosen harness and its extension model.	Delays commitment to a concrete agent architecture or runtime while the prototype evolves.
Development speed	Variable; fast for a narrow familiar system, slower as bespoke scope grows.	Can be fast when the base harness fits and needed extensions are straightforward.	Can be fast for adaptations that fit existing conventions; setup and constraints matter.	Aims to reduce the effort of prototyping and revising harnesses; comparative speed remains context-dependent.
Flexibility to adapt to change	Changes can be made broadly, but may require coordinated code and architecture changes.	Moderate and dependent on modularity and base-harness constraints.	Dependent on repository structure and coupling to the chosen harness.	Designed for flexible evolution before choosing a specific realization; not an assurance of lower cost for every change.
Primary goal	Deliver a domain-specific agentic application.	Make a general harness useful for a domain.	Adapt and evolve a particular harness for project use.	Rapidly prototype and evolve domain-specific harnesses before committing to a concrete system.

Interpretive note: The revfactory/harness column describes the repository-specific harness adaptation approach at a general level; its exact capabilities depend on the version and project configuration. The comparison is conceptual, not an empirical ranking.